



Architectural Solutions for Self-Adaptive Systems

Lina Garcés, University of São Paulo

Silverio Martínez-Fernández, Universitat Politècnica de Catalunya

Valdemar Vicente Graciano Neto, Federal University of Goiás

Elisa Yumi Nakagawa, University of São Paulo

Increasingly adopted in critical application domains, self-adaptive systems (SaSs) present a particular ability to modify their behavior or configuration at runtime autonomously. The architectural decision-making activity in an SaS can be supported by reusing proven software structure configuration.

Self-adaptive systems (SaSs) have increased their importance in the past few years, mostly because of their impact in several critical application domains, such as autonomous vehicles, smart cities, security surveillance, avionics, and health care. SaSs autonomously modify their

behavior at runtime, responding to operating environment changes.¹

An SaS is usually constituted by two types of systems:²

- › *managed systems*: those that monitor and affect the external world with which the SaS interacts and comprise the application logic that provides the system's functionalities

- › *managing systems*: those that encompass the adaptation logic that deals with one or more concerns, monitor both the environment and the managed systems, and adapt the latter when necessary to achieve SaS goals.

The managing system usually imposes control over the managed systems through the use of autonomic managers³ or monitor-analyze-plan-execute knowledge(MAPE-K) feedback loops that contain components of monitoring, analyzing, planning, executing, and sharing domain and control knowledge.⁴ During the architectural process, software architects must consider how control activities are coordinated and deployed, the most significant quality-attribute requirements (for example, performance, reliability, safety, availability, and scalability), and the type of adaptive capabilities required by the SaS,⁵ for instance, context awareness, situation awareness, self-configuration, self-healing/protecting, self-optimizing, self-managing, self-organizing, and reflection.

Adaptive capabilities allow decreasing human intervention when systems' modifications are required at runtime. Adaptive capabilities range from changes of specific data types to the reconfiguration of the complete SaS architecture in response to environmental changes, internal faults, unexpected constituents' behaviors, the integration of new constituents, new requirements, or changes in business goals. Architects must select the adequate strategies (for example, the architectural configurations, patterns, styles, tactics, or even technologies) that enable the desired modifications without stopping the system's operations and with minimal human

intervention. Architects' decisions making is a challenging activity during SaS engineering because the success or failure of an SaS software project mostly depends upon the correctness of its architecture for considering control activities, adaptive characteristics, and quality-attribute requirements. However, the rationale behind architectural decisions is frequently known by only the SaS software team (including the architect, developers, and testers), making it difficult to reuse of such knowledge in other SaS projects. In this context, one question arises: how can we support architects to design SaS architectures based on the reuse of architectural knowledge?

The main contribution of this article is the Four4SaS, a collection of four domain-independent solutions that guide and facilitate the architectural design of SaSs.

MINING SAS ARCHITECTURES

This section presents the research methods conducted to identify the main software building blocks of SaS architectures and their variation depending upon adaptive capabilities and control characteristics. This knowledge is the basis of the Four4SaS' architectural solutions. Bearing this goal in mind, we performed the following steps:

1. Recurrent architectural solutions were extracted from 13 existing reference architectures (RAs) for the SaS. A RA presents the most relevant decisions (for example, the selection and arrangement of patterns and tactics and a description of the rationale behind the proposed solutions) for designing

software systems architectures in specific domains.⁶ RAs for the SaS were identified through the conduction of a systematic mapping study following the guidelines found in the work of Petersen et al.⁷ The systematic mapping protocol and extracted data used in this work are detailed by Garcés.⁸ The interested reader is referred to this work,⁸ which contains IDs (RA1–RA13) and the complete reference for each architecture.

2. The recurrent SaSs' architectural solutions were extracted from 13 RAs for the SaSs listed in Garcés.⁸ The solutions were classified and summarized according to the main elements of an SaS architecture, namely, information about the SaSs' constituent systems, control characteristics, adaptive capabilities, and architectural patterns detailed in the "Constituents' Systems of SaSs," "Control Characteristics," "Requirements of Adaptive Capabilities," and "Architectural Patterns and Quality-Attribute Requirements" sections, respectively, and summarized in Table 1 and mentioned in the remainder of this section.

Constituents' Systems of SaSs

An SaS is formed from two types of constituent systems: managed and managing. Each constituent system can present diverse adaptive capabilities; hence, it can also be considered an SaS. This characteristic requires architectures with multiple adaptation levels, that is, an SaSs' composition is based on managing systems hierarchies,² such as those proposed by RA2, RA6, RA8, RA10, and RA12.

TABLE 1. The information extracted from RAs for SaSs.

Constituents of SaSs			Control characteristics of SaSs			Adaptivity requirements		
ID	Managing system	Managed system	Strategy	Control approach	Knowledge representation approach	Capabilities	Reflection	Human involvement
RA1	IPM system	IPM system	Category 1	Triggering conditions and events	Shared ontology between layers	Situation aware	—	—
RA2	Touchpoints and autonomic managers	Any self-adaptive IT system	Category 3	Hierarchical MAPE-K	Distributed and shared repositories between layers	Management	—	✓
RA3	Adaptive component	Any adaptive component	Category 1	Adaptive-component paradigm	Components' internal registers	Configuration	—	✓
RA4	Management SOA-based systems	Any SOA-based system	Category 4	Observer/controller architecture	Central knowledge repository	Organization	—	✓
RA5	Touchpoints and autonomic managers	Any component-based system	Category 1	MAPE-K	Central knowledge repository	Situation aware	—	✓
RA6	Reflection-based SaSs	Any SaS	Category 1	MAPE-K	Meta model	Configuration	✓	✓
RA7	Grid controller component	Microgrid	Category 3	Hierarchical MAPE-K	Shared ontology	Optimization	—	✓
RA8	Meta controller subsystem	Mobile robot's control application	Category 1	Epistemic control loop	Shared ontology	Configuration	✓	✓
RA9	Runtime environment	Runtime application	Category 2	Hierarchical feedback loop	Distributed and shared repositories between layers	Management	—	✓
RA10	Models@run.time system	Models@run.time system, cyberphysical system, or safety SaS	Category 1	MAPE-K	Multiple models	Configuration	✓	—
RA11	Enactors, managers, and solvers	System components' architecture	Category 2	Hierarchical MAPE-K	Central knowledge repository shared between layers	Management	✓	✓
RA12	Self-adaptive middleware	Wireless sensor network nodes	Category 2	Hierarchical MAPE-K	Repository by layer	Configuration	—	✓
RA13	HIIC component	Any SaS	Category 4	Hierarchical MAPE-K	Shared repository	Management	—	✓

SOA: service-oriented architecture; HIIC: hierarchical inter–intra collaborative; IPM: intelligent patient monitoring.

Control Characteristics

Control in SaSs is related to the distribution level of the managed and managing systems and the decentralization level of the control activities (that is, monitoring, analysis, planning, and execution).² Hence, the following

control strategies can be adopted to address adaptive capabilities in SaSs:

- › *Category 1 (C1) (The managed and managing systems and the control activities are centralized.):* This strategy is common in SaSs with

capabilities such as situation awareness (RA1 and RA5) and self-configuration (RA3, RA6, RA8, and RA10).

- › *Category 2 (C2) (The managed systems are distributed, while the managing systems are centralized*

and the control activities are decentralized.): This category supports capabilities such as self-configuration (RA12) and self-management (RA9 and RA11) in SaSs.

- › *Category 3 (C3) (Both managed and managing systems are distributed, and the control activities are decentralized.):* This strategy has been used in SaSs with capabilities of self-management (RA2) and self-optimization (RA7).
- › *Category 4 (C4) (Both managed and managing systems are distributed, and the control activities are fully decentralized.):* SaSs with capabilities of self-organization can be addressed using this control category (RA4 and RA13).

The implementation of these control strategies is mostly based on the MAPE-K loop and its variations, as detailed in Table 1. To represent domain and control knowledge, repositories, ontologies, and conceptual models are commonly used by the RAs to represent such knowledge.

Requirements of Adaptive Capabilities

Architectural solutions proposed in RAs focus on adaptive capabilities, such as situation awareness, self-configuration, self-management, and self-organization. Reflection is considered in SaSs' architectures with self-management and self-configuration characteristics. Most of the RAs (that is, eight out of 13) consider the occurrence of changes, simultaneously, in both managed and managing systems. Different SaSs' structures can change, ranging from managed systems' entities (for example, components, services, or interfaces) to managing systems' plans, policies, and goals.

No human involvement allows for execution of close adaptations (as in RA1 and RA10), that is, the SaSs themselves manage a number of predefined adaptive actions, and no new behaviors and alternatives can be introduced at runtime.⁵ Moreover, most architectures define open adaptations, that is, new goals, requirements, and policies can be added, and even new adaptable entities can be introduced by humans.⁵

Architectural Patterns and Quality-Attribute Requirements

The most-employed architectural patterns to design SaSs and address diverse adaptivity and quality-attribute requirements are described as follows.

Layers have been widely used regardless of the desired adaptive capabilities. Layers enable complex behaviors through hierarchies (RA1, RA3, RA4, RA5, and RA11). Lower layers implement fast adaptations (that is, reconfigurations of the managed system), and higher layers are responsible for time-demanding adaptations (that is, the selection of the best policy or plan to achieve missions based on the current system status). Lower layers in RAs achieve performance requirements (RA1), and higher layers address reliability properties (RA10). Interoperability can also be supported by establishing generic connections between managed systems and managers (RA2 and RA5). Layers' separation of concerns enhances maintainability and modifiability requirements.⁹

A shared-data repository is commonly presented in SaSs' architectures with self-configuration, self-management, or self-organization capabilities. It allows access to persistent data, ensuring the availability of context, control, configuration, and domain information. Also, it avoids undesired

changes of data due to modifications in managed and managing systems, contributing to the modifiability⁹ of SaSs (RA2, RA4, RA9, RA11, and RA12). Finally, repositories reduce the data exchange between managed and managing systems and between the hierarchies of these systems.

A blackboard and pipes and filters have been applied in SaSs' architectures with situation-awareness capability (RA1 and RA5). Together, both patterns benefit performance, as the blackboard allows for the efficient delivery of data between system layers,⁹ and the pipe and filter patterns grant the concurrent execution of adaptations by high-level layers.⁹

Publish and subscribe is used in SaSs' architectures for self-optimization (RA7) with the following benefits: 1) performance, to communicate data among SaSs' entities; 2) modifiability, due to low coupling between entities; and 3) dynamic scalability, as managed systems can enter or exit without affecting other parts of the SaS.

A service-oriented architecture (SOA) has been applied in SaSs with capabilities of self-organization and self-configuration (RA4 and RA12). SOA benefits are 1) the interoperability of managed systems, 2) the evolution and dynamic scalability of SaSs according to the new demands of resources, and 3) the availability of managing systems through their replication for processing the monitored data of managed systems. The combination of an SOA and a enterprise service bus (ESB) facilitates the integration of multiple managed systems, mediating and transferring monitored data to all of the managing systems interested in such data.

A master-slave relationship has been adopted in SaSs' architectures

with self-management properties (RA11). It benefits performance by providing low response times to control changes in the managed system.

A mediation layer facilitates communication, interoperability, and integration of heterogeneous managed systems. Brokers have been considered for SaSs with self-configuration requirements (RA12).

A decorator pattern enables behavior adaptations at the fine-granularity level, for example, adaptations of components objects or parameters values. Decorators can improve the reliability of adaptations in SaSs with self-configuration properties (RA12).

A hierarchical inter-intra collaborative (HIIC) proposes hierarchies of MAPE-K loops and their coordination to make possible a decentralized control in SaSs.¹⁰ An HIIC has been used in SaSs with high-level adaptive capabilities, such as self-configuring or self-management (RA13).

FOUR4SAS: ARCHITECTURAL SOLUTIONS FOR SASS

Adaptive capabilities and distributed levels of control and constituent systems have a strong influence on determining SaSs' architectures. Four4SaS defines four generic and reusable solutions based on the architectural knowledge mined from RAs for SaSs. Table 2 summarizes each Four4SaS solution, detailing information about

- › the control strategies adopted by the solutions (C1–C4) and mentioned in the “Architectural Solution for SaSs With a Control Strategy of C1,” “Architectural Solution for SaSs With a Control Strategy of C2,” “Architectural Solution for SaSs With a Control Strategy of C3,” and

“Architectural Solution for SaSs With a Control Strategy of C4” sections, respectively

- › the monitored elements in the SaS (that is, the managed- or manager systems layer)
- › the reasons ([R]) why adaptations are required
- › the SaS elements ([E]) that need to be adapted
- › the adaptive requirements that can be addressed with Four4SaSs' solutions
- › the adaptation type, depending on whether the architecture allows open or close adaptations
- › the benefits ([B]) regarding the quality-attribute requirements and capacities achieved with the solutions
- › the possible drawbacks ([D]) that the solutions could bring to an SaS.

In Table 2, in the cases where the reasons for adaptations ([R]), elements to be adapted ([E]), benefits ([B]), and drawbacks ([D]) are presented in more than one solution, they are described on their first occurrence, and only their codes are used in the text. Architects can use Table 2 as a guide to select the adequate solution for architecting an SaS. Each solution is depicted in Figure 1 and explained as follows.

Architectural Solution for SaSs With a Control Strategy of C1

To design SaSs with the adaptive requirements of situation aware and self-configuration, architects can structure their systems, as shown in Figure 1(a). This architecture is adequate for SaSs that, based on measurements obtained from the managed system, require making changes to the adaptation plans or policies, or even in

the behavior, state, or configuration of such systems. SaS modifications occur when the final user (the SaS administrator) updates the adaptation plans and policies, faults are detected, or managed systems change their state and configuration. This architecture enables close adaptations, that is, the manager system (for example, the autonomic manager) decides which type of reconfiguration should be executed based on policies or plans previously stored in a shared repository.

The one benefit shared by all of Four4SaSs' solutions is the enhancement of modifications and the maintenance of SaS elements thanks to the low coupling that layers provide. This first solution also makes it possible to execute a fast reconfiguration of the managed system when the control activities (that is, MAPE) are organized as a centralized MAPE-K feedback loop that follows the pipes and filters pattern. Fast communications are also possible when the blackboard pattern (allocated in the repositories layer) is used to store and communicate all of the measures obtained from the managed systems.

A drawback of this solution is that the SaS will have a monolithic and nonscalable architecture. Besides, the existence of a unique autonomic manager can result in a single point of failure that could completely stop the SaSs' operations. Additionally, the centralized control made by the autonomic manager only allows for addressing an individual adaptive property at the time, limiting the type of adaptations that an SaS can perform.

Architectural Solution for SaSs With a Control Strategy of C2

The second solution of the Four4SaS [presented in Figure 1(b)] allows for the

TABLE 2. A description of Four4SaS architectural solutions.

Four4SaSs' solution	Monitored elements	Reason for adaptation	Elements to be adapted	Adaptivity requirements	Adaptation type	Benefits	Drawbacks
Control strategy C1	Managed-systems layer	[R1] New adaptation plans or policies [R2] Undesired situations detected [R3] The installation, updating, and integration of systems or components	[E1] Adaptation plans or policies [E2] The behavior, states, or configuration of the managed systems	Situation aware and the self-configuration of managed systems	Closed adaptations	[B1] Fast SaS adaptations [B2] Fast communication of SaS situations or events [B3] The easy maintainability or modifiability of components	[D1] Monolithic architecture [D2] No scalable architecture [D3] Managers can be a single point of failure [D4] Oriented to address individual adaptive capabilities
Control strategy C2	Managed and managers' systems layers	[R1], [R3] [R4] Fault discovery or diagnosis	[E1], [E2] [E3] The behavior, states, or configuration of systems in the managers' layer	Self-configuration and self-management of the SaS	Closed and open adaptations	[B3] [B4] Systems at the managed-systems layer can scale [B5] The redundancy of monitored or managed systems [B6] The hierarchies of managers' systems allow for complex SaS reconfigurations or adaptations	[D3], [D4] [D5] The mediation layer can be a single point of failure [D6] High-level managers are not scalable
Control strategy C3	Managed and managers' systems layers	[R1], [R4] [R5] Management of performance and resource allocation	[E1] – [E3] [E4] Mediation layers	Self-optimization and self-management of the SaS	Closed and open adaptations	[B3] – [B6] [B7] Enable the identification, prevention, and recovery from faults at all layers due to problems in physical nodes	[D4] – [D6] [D7] Possible bottleneck in the mediation layer due to the increment of data to be transferred
Control strategy C4	Managed and managers' systems layers	[R1] [R6] Instantiate, activate, deactivate, remove, and update the elements located in the managers' systems layer [R7] Scale the elements in the managers' systems layer	[E1] – [E4]	Self-organization and scalability of the SaS	Closed and open adaptations	[B3] and [B7] [B8] Various high-level managers can address multiple adaptive capabilities at the same time [B9] Monitored data can be distributed to multiple managers for different purposes [B10] Systems at the managed and managers' systems layers can scale	[D5], [D7] [D8] Possible conflict between adaptive properties [D9] Additional strategies for coordinating highly distributed autonomic managers located in the managers' systems layer

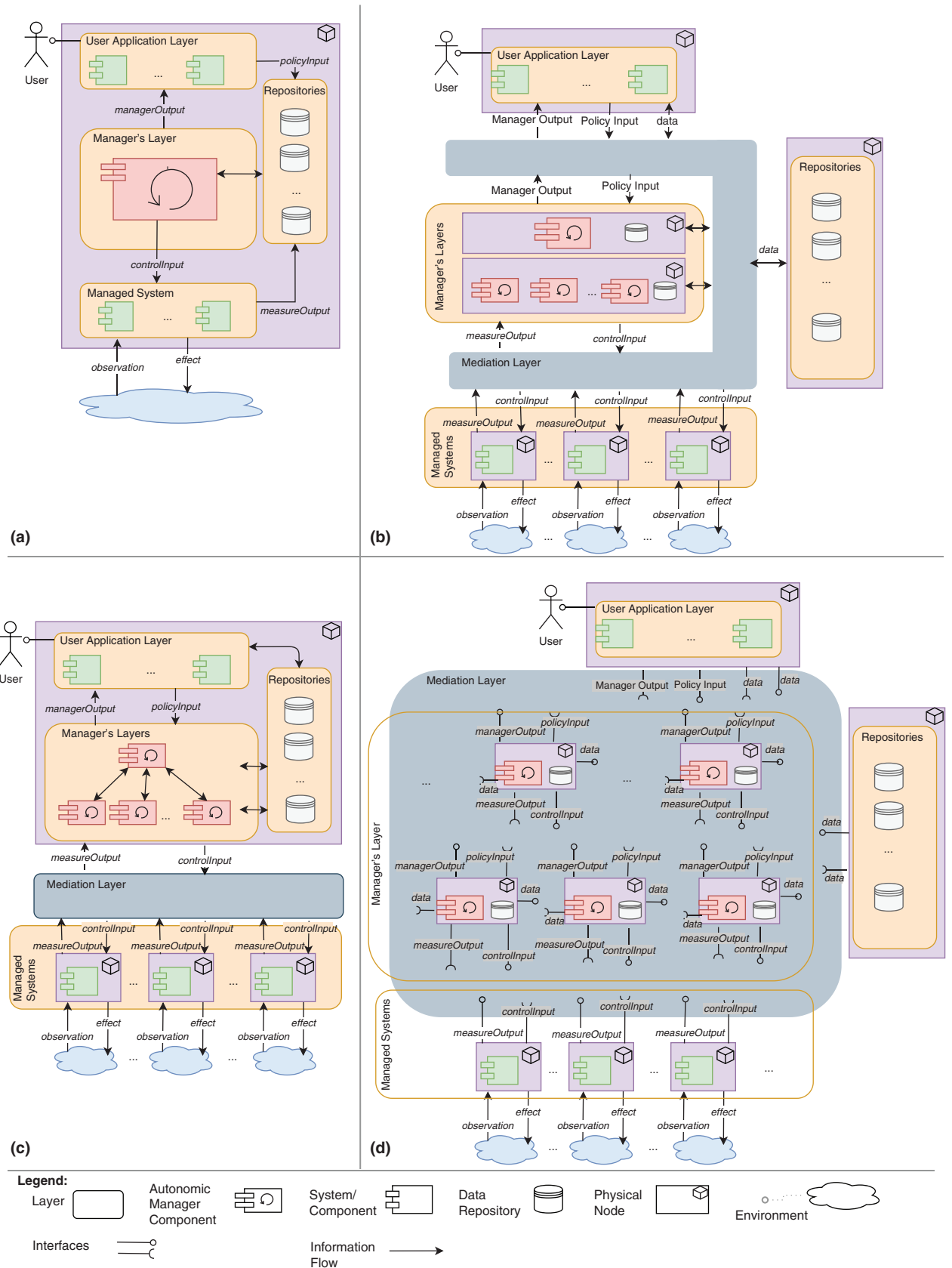


FIGURE 1. An architectural solution for SaaS with control strategies (a) C1, (b) C2, (c) C3, and (d) C4.

adaptive capabilities of self-configuration and self-management. Compared to the previous alternative, this solution requires the monitoring of both managed and manager systems. The information obtained from monitored system is used by high-level managers to define which elements to change, that is, the behavior, state, and configuration of managed systems and low-level managers due to faults presented by the monitored systems. It is possible to execute close adaptations in an SaS. Open adaptations are also possible because final users (SaS administrators) can send new policies directly to high-level managers. The distributed property of managed systems requires a mediation layer to support data transfer between managed systems and managers.

One benefit of this architecture is the possibility of scaling and replicating managed systems and low-level managers, increasing the reliability of monitored data. This is possible from the allocation of managed systems in distributed physical nodes. Additionally, the distribution of managers, following a hierarchy of MAPE-K components, allows for the planning and execution of more complex adaptations by high-level autonomic managers. This characteristic also favors the SaSs' reliability. In this architecture, fast changes are possible when the master-slave relationship or the decorator patterns are used as a basis to organize MAPE-K hierarchies. The combination of broker and publish-subscribe patterns can be used to structure the mediation layer, improving interoperability between managed systems and managers and the performance when data are transferred.

One drawback is that the mediation layer can be a single point of failure,

disconnecting the managed systems and stopping the SaSs' operation. Moreover, managers at a high level are not scalable, hindering the achievement of multiple adaptive properties.

Architectural Solution for SaSs With a Control Strategy of C3

This solution aims at both self-optimization and self-management capabilities. As shown in Figure 1(c), the managed and managing systems are distributed. They can be modified at runtime and, together with the mediation layer, are the elements being monitored. The hierarchies of MAPE-K components are allocated in different nodes; hence, control activities are decentralized. This characteristic benefits the execution of complex reconfiguration and the management of SaS performance and resource-allocation capabilities. This architecture presents benefits similar to the previous solutions. An additional benefit is the identification, prevention of faults, and recovery from the failures caused by the physical decoupling of MAPE-K hierarchies. This property increases the SaSs' reliability. An important drawback is the possible bottleneck in the mediation layer due to the increase of the transferred data.⁹

Architectural Solution for SaSs With a Control Strategy of C4

The last strategy distributes and monitors both managed systems and managers. Hence, SaSs with requirements of high scalability, flexibility to add/activate/deactivate/update elements, and self-organization can use this solution as a blueprint for their architecture.

This solution benefits diverse types of adaptations with distinct complexity performed by fully decentralized managers deployed in multiples nodes, following the hierarchical MAPE-K or HIIC¹⁵ patterns. As a complement, the use of an SOA, a broker, or an ESB favors the scalability⁹ of managed systems and managers in an SaS. These characteristics differentiate this solution from the previous three, as depicted in Figure 1(d).

This solution is the most complete among the Four4SaS solutions, enhancing the quality attributes of maintainability, reliability, and interoperability, as listed in Table 2. A possible drawbacks is related to the simultaneous execution of multiple adaptive requirements, which can generate conflicts.^{5,11} This drawback can be overcome through the coordination of highly distributed and by hierarchical managers.¹¹ Additionally, the use of a mediation layer to communicate data, events, and control can generate problems in the performance and reliability of the SaS.

APPLYING FOUR4SAS: THE RIVER-MONITORING SYSTEM CASE

A river-monitoring system (RMS) is responsible for monitoring river levels in a given region. It consists of systems/devices, such as motes, gateways, river monitors, sensor-observation services, web map services, and emergency services. Motes and gateways are distributed over the river banks. Each mote measures and communicates its observations (for example, water level, temperature, pressure, or pollutants) or its configuration information (for instance, the description, identification, and classification of the mote) to the closest

gateway. At runtime, gateways establish specific river-area situations, detect unexpected events (for example, floods in a region), define plans to overcome problematic situations (for example, to restart unavailable motes), and execute such plans. Gateways share a central repository containing control information, such as adaptation plans, emergency situations, mote network configurations, and policies. Gateways communicate information about the river areas and system situations to river-monitoring components, which aggregate the information received from all the gateways to establish an overall panorama of the river and send this information to the user applications. The river monitor receives reconfiguration requests from user applications (for example, requesting water and temperature levels of the entire or of parts of the river) and communicates new behavior or configuration policies to gateways and motes. More information about RMSs can be found in the work of Degrossi et al.¹²

Selecting a Four4SaS Architectural Solution

Before selecting a Four4SaS solution, the RMS must be characterized according to the SaS properties defined in the “Four4SaS: Architectural Solutions for SaSs” section and Table 2:

- › **Monitored elements:** This means that motes and gateways are both distributed over the environment (namely, the river).
- › **Managers:** This role is taken by gateways (physically distributed) and river monitors (both possibilities exist, that is, they can be either centralized or distributed).

- › The following are reasons for adaptations in the RMS:
 - › **Interoperability and flexibility:** It is required to add, remove, instantiate, deactivate, and activate motes and gateways without requiring manual adaptations.
 - › **Scalability:** It is required to continuously add more motes and gateways to cover new river(s) areas independently of the city.
 - › **Reliability:** It is important to identify when motes and gateways are unavailable (for example, as a result of hardware problems). Additionally, it is required to detect, prevent faults, and recover from faults that occur in motes and gateways.
 - › **Maintainability:** Modifications in policies, requirements, adaptation plans, or even in individual motes and gateways must not affect the RMSs’ operation.
- › The following elements are to be adapted:
 - › adaptation plans, policies, and requirements
 - › motes’ behaviors, states, and configuration (for example, the measurement rates of water levels)
 - › gateways’ behaviors, states, and configuration
 - › the capacity for supporting multiple connections and a large amount of data transferred among the systems.
- › **The type of adaptation:** The RMS requires adopting close adaptations in motes, gateways, and open adaptations (involving final users) to make possible

new policies of reconfiguration (for example, an increase in the water-level measurement rates of motes in a region).

Therefore, the RMSs’ architecture needs to address the adaptive capabilities of self-configuration and self-organization. Considering that multiple adaptive properties are necessary, managed systems and managers are distributed, and scalability is an important requirement of this SaS, as it requires the full decentralization of autonomic managers’ control activities. Upon analyzing the Four4SaS solutions presented in Table 2, the best option for the RMSs’ architecture is to follow the solution of control strategy C4. Figure 2 presents the final RMS architecture as an example of this solution.

FOUR4SAS ASSESSMENT

To assess Four4SaSs’ quality characteristics (its usefulness) to design software architectures of SaSs, we ran an evaluation with architects who could use Four4SaS in their projects. They were trained in the Four4SaS and answered an online questionnaire (available in the work of Garcés⁸) based on the technology acceptance model (TAM) evaluation questionnaire.¹³ The questionnaire included validated constructs from the TAM to evaluate five criteria: usefulness, ease of use, demonstrability, feasibility, and the quality of architectures created with Four4SaS. An example of using an SaS architecture with Four4SaS was discussed during the training, specifically, the RMS presented in the “Applying Four4SaS: The River-Monitoring System Case” section.

Fifteen architects participated in this study. They had two–eight years ($N = 15$, minimum = 2, maximum = 8, and

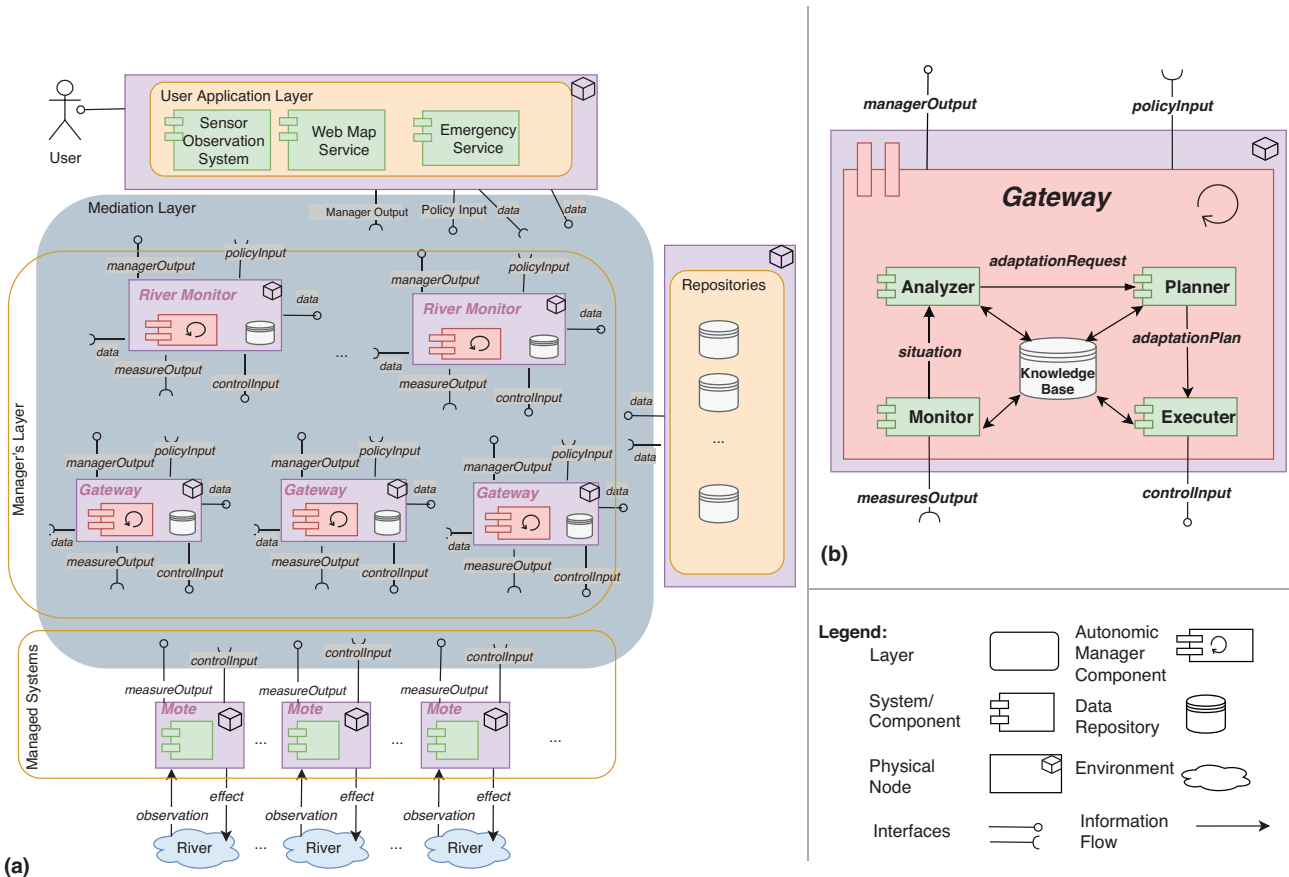


FIGURE 2. The RMS architecture as an instance of the Four4SaS architectural solution with control strategy C4. (a) The general view of the RMS architecture. (b) The internal view of a gateway as an autonomic manager.

mean = 4) of experience working with SaSs for different application domains, including the Internet of Things embedded, health care, crisis and emergency, robotics, banking, and spacial systems.

The answers to each question were scored from 1 (strongly disagree) to 7 (strongly agree). Table 3 summarizes the results of the Four4SaS assessment. For each question, the number of answers scored in a specific value (from 1 to 7) are given. The score tendencies are highlighted on a gray-color scale (a cell in white represents that no architect scored a question with such a value). For each criterion,

the aggregated results are presented, detailing the mean, standard deviation, median, mode, and minimum and maximum scores given by the respondents.

The majority of the responses positively scored all the evaluation criteria. We only obtained a negative result for the feasibility criterion, regarding "All of the information necessary to understand Four4SaS is available." Fifty-three percent (eight out of 15) of the architects answered this question with a score of lower than or equal to three, for a median of three (somewhat disagree).

We also provided the possibility for respondents to include free text. The main responses from the architects were 1) systematizing the selection of Four4SaSs' solutions and supporting architectural decision making using automated tools; 2) offering metrics pertaining to the possible tradeoffs of using Four4SaSs' solutions in specific projects; 3) improving Four4SaSs' descriptions using different architectural views, implementation details, and a website to link additional information; 4) automatic code generation based on Four4SaSs' architectures to support

TABLE 3. The criteria, questions, and answers used to evaluate the Four4SaS. The number of participants: 15. The answers were rated on a scale from 1 to 7: 1—strongly disagree, 2—moderately disagree, 3—somewhat disagree, 4—neutral (neither disagree nor agree), 5—somewhat agree, 6—moderately agree, and 7—strongly agree. The questions marked with (*) were asked of architects in their negative form. Herein, they are presented in their positive form to facilitate the analysis of the results.

Criteria	Question	Number of answers by scale						
		1	2	3	4	5	6	7
Perceived usefulness	Using Four4SaS could improve my performance in my job/research.	0	0	0	0	3	9	3
	Using Four4SaS in my job/research could increase my productivity.	0	0	0	1	4	5	5
	Using Four4SaS could enhance my effectiveness in my job/research.	0	0	0	3	7	4	1
	I find that Four4SaS could be useful in my job/research.	0	0	0	1	5	6	3
	Aggregated results by criterion: mean = 6, standard deviation = 0.87, median = 6, mode = 6, minimum = 4, and maximum = 7							
Perceived ease of use	I could use Four4SaS in a clear and understandable way.	0	1	1	0	3	6	4
	Using Four4SaS would not require a lot of my mental effort.	0	1	0	2	7	5	0
	I find Four4SaS to be potentially easy to use.	0	0	1	3	4	4	3
	I find it is easy to get from Four4SaS the knowledge to construct new SaS architectures.	0	0	0	3	6	5	1
	Aggregated results by criterion: mean = 5, standard deviation = 1.16, median = 5, mode = 5, minimum = 2, and maximum = 7							
Result demonstrability	I have no difficulty explaining to others the benefits of using Four4SaS.	0	0	1	2	3	4	5
	I believe I could communicate to others the consequences (drawbacks) of using Four4SaS.	0	0	0	1	2	6	6
	The results of using Four4SaS are clear to me.	0	0	1	0	3	8	3
	I would have no difficulty explaining why using Four4SaS may or may not be beneficial (*).	1	2	0	3	2	6	1
	Aggregated results by criterion: mean = 6, standard deviation = 1.3, median = 6, mode = 6, minimum = 1, and maximum = 7							
Feasibility	All of the information necessary to understand Four4SaS is available (*).	2	3	3	2	0	4	1
	Making available more information, even if possible, would not be too costly (*).	0	1	0	5	5	2	1
	The quality of the information provided to understand Four4SaS is good (*).	0	0	3	1	3	2	6
	All necessary information to fully understand Four4SaS can be used (*).	0	0	0	5	2	4	4
	Aggregated results by criterion: mean = 5, standard deviation = 1.6, median = 5, mode = 4, minimum = 1, and maximum = 7							
Quality of architectures created with Four4SaS	The quality of the RMS architecture obtained from Four4SaS can be considered as high.	0	0	0	1	3	10	1
	I have no problem with the quality of the RMS architecture.	0	0	0	3	5	5	2
	I rate as excellent the results from using Four4SaS to create an RMS architecture.	0	0	0	3	1	9	2
	Aggregated results by criterion: mean = 6, standard deviation = 0.87, median = 6, mode = 6, minimum = 4, and maximum = 7							

ABOUT THE AUTHORS

LINA GARCÉS is a postdoctorate researcher with the Institute of Mathematics and Computer Sciences at the University of São Paulo (USP). Her research interests include software engineering, (dynamic) software architectures, reference architectures, architectural decision making, and software quality. Garcés received a Ph.D. in computer science from USP and the University of Southern Brittany, France. She is a Member of IEEE and the IEEE Standards Association. Contact her at linamgr@icmc.usp.br.

SILVERIO MARTÍNEZ-FERNÁNDEZ is an assistant professor at the Universitat Politècnica de Catalunya (UPC)-BarcelonaTech. His research interests include empirical software engineering, big data, technical debt, maintainability metrics, and software architectures. Martínez-Fernández received a Ph.D. in computer sciences from UPC. Contact him at smartinez@essi.upc.edu.

VALDEMAR VICENTE GRACIANO NETO is a tenured professor at the Informatics Institute of the Federal University of Goiás, Brazil. His research interests include software engineering, information systems, software architecture, model-driven software engineering, and simulation. Neto received a Ph.D. in computer science from the Universitat Politècnica de Catalunya and the University of Southern Brittany, France. Contact him at valdemarneto@inf.ufg.br.

ELISA YUMI NAKAGAWA is an associate professor in the Department of Computer Systems at the Universitat Politècnica de Catalunya (USP). Her research interests include software architecture, reference architectures, systems-of-systems, software testing, and evidence-based software engineering. Nakagawa received a Ph.D. in computer science from USP. She is a Member of the IEEE. Contact her at elisa@icmc.usp.br.

researchers for use in different projects, making possible the identification of Four4SaSs' variations and possibly complementary solutions.

Several strategies for designing SaS architectures have been proposed during the last decade. Most of them are focused on investigating feedback loops (such as MAPE-K) as core elements of SaS architectures.^{14–17} Researchers also have proposed some patterns for self-adaptation^{18,19} and runtime software evolution,²⁰ some of which are oriented to SOA-based SaSs.¹⁹

Four4SaSs' solutions advance the state of the art by bringing to light reusable architectural knowledge about how to design SaSs considering variations of adaptive capabilities, quality-attribute requirements, the necessity of human involvement to perform open or closed adaptations, and the distribution level of SaS constituents (that is, managed and managing systems) and control operations. Four4SaSs' solutions are based on evidence obtained from 13 published SaS RAs, which are listed in the work of Garcés.⁸

As future work, Four4SaSs' solutions could be formalized in an SaS architectural framework to guide decision making and enable automatic code generation of SaSs. More efforts must be also invested to better understand the tradeoffs arising from addressing simultaneously multiple adaptive characteristics in SaS architectures (for example, following the C4 strategy of Four4SaS). ■

ACKNOWLEDGMENTS

This work was supported by Brazilian funding agencies FAPESP (Grants 2018/07437-9 and 2017/06195-9) and CNPq (Grant 312634/2018-8) and by the

productivity; and 5) using Four4SaS as a basis for product-line architectures in specific SaS domains, for example, the Internet of Things.

LIMITATIONS

Threats to the search, data, and research validity of our work were mitigated by following the guidelines for conducting a systematic mapping study⁷ and executing an online questionnaire with SaS architects following the TAM approach.¹³ The work of Garcés⁸ presents

the protocol, execution, and extracted data of our systematic mapping study as well as the form and results of conducting the TAM questionnaire.

The main limitation of Four4SaS is that its architectural solutions are described at a higher abstraction level. Hence, architects need to refine Four4SaSs' structures to achieve more detailed architectures of their concrete SaS projects. To overcome this limitation, Four4SaS must be also disseminated to SaS practitioners and

GENESIS Spanish project under Contract TIN2016-79269-R and the Beatriz Galindo programme. The authors thank all of the researchers and practitioners who participated in the evaluation study.

REFERENCES

1. P. Oreizy et al., "An architecture-based approach to self-adaptive software," *IEEE Intell. Syst.*, vol. 14, no. 3, pp. 54–62, 1999. doi: 10.1109/5254.769885.
2. D. Weyns et al., "On patterns for decentralized control in self-adaptive systems," in *Software Engineering for Self-Adaptive Systems II* (Lecture Notes in Computer Science), vol. 7475, R. de Lemos, H. Giese, H. A., Müller, and M. Shaw, Eds. Berlin: Springer-Verlag, 2013; pp. 76–107.
3. "An architectural blueprint for autonomic computing," IBM Corp., Armonk, NY, White Paper, 2005. [Online]. Available: <https://www-03.ibm.com/autonomic/pdfs/AC%20Blueprint%20White%20Paper%20V7.pdf>
4. H. A. Müller, H. M. Kienle, and U. Stege, "Autonomic computing now you see it, now you don't," in *Software Engineering* (Lecture Notes in Computer Science), vol. 5413, A. de Lucia and F. Ferrucci, Eds. Berlin: Springer-Verlag, 2009, pp. 32–54.
5. M. Salehie and L. Tahvildari, "Self-adaptive software: Landscape and research challenges," *ACM Trans. Autonom. Adapt. Syst.*, vol. 4, no. 2, pp. 1–42, 2009. doi: 10.1145/1516533.1516538.
6. E. Guerra and E. Y. Nakagawa, "Relating patterns and reference architectures," in *Proc. 22nd Conf. Pattern Languages Programs*, 2015, pp. 1–9. doi: 10.5555/3124497.3124507.
7. K. Petersen, S. Vakkalanka, and L. Kuzniarz, "Guidelines for conducting systematic mapping studies in software engineering: An update," *Inf. Softw. Technol.*, vol. 64, pp. 1–18, Aug. 2015. doi: 10.1016/j.infsof.2015.03.007.
8. L. Garcés, 2020, "Research on Reference Architectures for Self-Adaptive Systems: A Systematic Mapping Protocol and Extracted Data," Mendeley Data. [Online]. Available: <https://data.mendeley.com/datasets/w55h5cwyzc/draft?a=c9f71899-d8aa-494d-964a-9a717709dd5f>
9. P. Avgeriou and U. Zdun, "Architectural patterns revisited—a pattern language," in *Proc. 10th Euro. Conf. Pattern Languages of Programs*, 2005, pp. 1–39.
10. E. Zavala, X. Franch, J. Marco, A. Knauss, and D. Damian, "SACRE: Supporting contextual requirements' adaptation in modern self-adaptive systems in the presence of uncertainty at runtime," *Expert Syst. Appl.*, vol. 98, pp. 166–188, May 2018. doi: 10.1016/j.eswa.2018.01.009.
11. D. Weyns, "Software engineering of self-adaptive systems: An organised tour and future challenges," in *Handbook of Software Engineering*, S. Cha, R. Taylor, and K. Kang, Eds. Berlin: Springer-Verlag, 2019, pp. 399–443. doi: 10.1007/978-3-030-00262-6_11.
12. L. C. Degrossi, G. G. do Amaral, E. S. M. de Vasconcelos, J. P. de Albuquerque, and J. Ueyama, "Using wireless sensor networks in the sensor web for flood monitoring in Brazil," in *Proc. 10th Int. ISCRAM Conf.*, 2013, pp. 458–462.
13. F. F. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quart.*, vol. 13, no. 3, 1989. doi: 10.2307/249008.
14. D. Weyns and T. Ahmad, "Claims and evidence for architecture-based self-adaptation: A systematic literature review," in *Proc. European Conf. Software Architecture*, 2013, pp. 249–265. doi: 10.1007/978-3-642-39031-9_22.
15. E. Zavala, X. Franch, J. Marco, and C. Berger, "HAFLoop: An architecture for supporting Highly Adaptive Feedback Loops in self-adaptive systems," *Future Gener. Comput. Syst.*, vol. 105, pp. 607–630, Apr. 2020. doi: 10.1016/j.future.2019.12.026.
16. H. Muccini, M. Sharaf, and D. Weyns, "Self-adaptation for cyber-physical systems: A systematic literature review," in *Proc. 2016 IEEE/ACM 11th Int. Symp. Software Engineering for Adaptive and Self-Managing Systems*, pp. 75–81. doi: 10.1145/2897053.2897069.
17. P. Arcaini, R. Mirandola, E. Riccobene, and P. Scandurra, "A pattern-oriented design framework for self-adaptive software systems," in *Proc. 2019 IEEE Int. Conf. Software Architecture Companion*, pp. 166–169.
18. H. Muccini, R. Spalazzese, M. T. Moghaddam, and M. Sharaf, "Self-adaptive IoT architectures: An emergency handling case study," in *Proc. 12th European Conf. Software Architecture*, 2018, pp. 1–6. doi: 10.1145/3241403.3241424.
19. H. Gomaa, K. Hashimoto, M. Kim, S. Malek, and D. A. Menascé, "Software adaptation patterns for service-oriented architectures," in *Proc. 2010 ACM Symp. Applied Computing*, pp. 462–469. doi: 10.1145/1774088.1774185.
20. R. N. Taylor, N. Medvidovic, and P. Oreizy, "Architectural styles for runtime software adaptation," in *Proc. 2009 Joint Working IEEE/IFIP Conf. Software Architecture & European Conf. Software Architecture*, pp. 171–180.